

IV

MANEJO DEL ESTADO CON PROVIDER

A medida que las aplicaciones Flutter se vuelven más complejas, gestionar eficientemente el estado se convierte en un aspecto clave para mantener el código limpio, modular y mantenible. En este capítulo, se centrará el estudio en el uso de Provider, uno de los enfoques más populares y efectivos para manejar el estado en aplicaciones Flutter a gran escala. Provider no solo simplifica la gestión del estado, sino que también facilita la inyección de dependencias, resultando en un flujo de datos más claro y una arquitectura de aplicación más coherente.

Se comenzará con una introducción a los conceptos básicos de Provider, explicando cómo puede ayudar a separar la lógica de negocio de la interfaz de usuario, lo que permite una mayor flexibilidad y reutilización del código. A través de ejemplos prácticos, se verá cómo estructurar la aplicación para aprovechar las ventajas de Provider, incluyendo la creación de modelos de datos reactivos y la actualización eficiente de la UI en respuesta a cambios en el estado.

El ejercicio práctico de este capítulo guiará a través de la implementación de Provider en la aplicación de lista de planes que se ha estado desarrollando, este paso permitirá aplicar los principios de gestión de estado avanzada, observando de primera mano cómo Provider facilita la manipulación de datos y estados a través de múltiples widgets y pantallas de manera eficiente y escalable.

4.1 Manejo del estado con Provider

Remi Rousselet diseñó Provider para envolver *InheritedWidget*, simplificándolo, Google ya había creado su propio paquete para gestionar el estado, pero se dieron cuenta de que Provider era mejor, ahora recomiendan utilizarlo en su lugar, incluso Google incluye detalles al respecto en su documentación sobre gestión de estado³⁵.

Provider es un patrón de diseño que envuelve la arquitectura *InheritedWidget* de Flutter, la cual es compleja y verbosa de usar directamente. Provider simplifica este proceso, haciendo más fácil y limpio el manejo del estado a través de la aplicación.

Provider destaca por las siguientes características:

- **Simplicidad:** Ofrece una API más simple para pasar datos a lo largo del árbol de widgets.
- **Rendimiento:** Ayuda a optimizar el rendimiento al evitar la reconstrucción innecesaria de widgets.
- **Flexibilidad:** Se adapta a diferentes arquitecturas y patrones de manejo de estado.

4.1.1 Clases usadas por Provider

Provider tiene varias clases comúnmente utilizadas: *ChangeNotifier*, *ChangeNotifierProvider*, *Consumer*, *FutureProvider* y *MultiProvider*. Se explicará cada una de ellas en los párrafos siguientes:

ChangeNotifier

ChangeNotifier es una clase que facilita la gestión de estados permitiendo la adición y eliminación de oyentes, así como la notificación a estos oyentes sobre cualquier cambio. Se suele extender esta clase en los modelos para emitir notificaciones tras cada modificación. De este modo, al producirse un cambio y al invocar a *notifyListeners()*, se desencadena una actualización en cualquier widget que esté escuchando. Esto permite la utilización del estado actualizado del modelo para redibujar partes específicas de la interfaz de usuario de manera eficiente. A continuación, se presenta un modelo que extiende la clase *ChangeNotifier*:

```
class MyModel extends ChangeNotifier {  
  ...  
}
```

ChangeNotifierProvider

ChangeNotifierProvider es un widget diseñado para envolver una clase que implemente *ChangeNotifier*, sirviendo como puente para la comunicación de cambios. Este widget utiliza un widget hijo especificado para la visualización de la interfaz de usuario, cuando ocurren cambios en el estado gestionado por *ChangeNotifier*, el *ChangeNotifierProvider* reacciona reconstruyendo su árbol de widgets para reflejar los cambios. La estructura para implementar este patrón se presenta de la siguiente manera:

³⁵ <https://docs.flutter.dev/data-and-backend/state-mgmt/simple>

```
ChangeNotifierProvider(  
  create: (context) => MyModel(),  
  child: <widget>,  
);
```

El proveedor ofrece un punto central para gestionar el estado, de modo que no necesites actualizar manualmente diferentes widgets mediante `setState()` cada vez que el estado cambia.

Consumer

El widget *Consumer* está diseñado para escuchar cambios en cualquier clase que implemente *ChangeNotifier*, procediendo a reconstruir los widgets bajo su jerarquía ante cualquier modificación detectada.

En la construcción de la jerarquía de widgets, es recomendable situar el *Consumer* en un nivel lo más profundo posible dentro de la estructura de la interfaz de usuario. Este enfoque minimiza la necesidad de reconstruir extensas porciones del árbol de widgets, optimizando así el rendimiento de la aplicación al limitar las actualizaciones solo a las áreas afectadas por los cambios en el estado.

Se sugiere observar el siguiente ejemplo para obtener claridad sobre la sintaxis.

```
Consumer<MyModel>(  
  builder: (context, model, child) {  
    return Text('Hello ${model.value}');  
  }  
);
```

Si se requiere acceso al modelo sin necesidad de recibir notificaciones ante cambios en los datos, se debe utilizar *Provider.of* de la siguiente manera:

```
Provider.of<MyModel>(context, listen: false).<method name>
```

El parámetro *listen: false* indica que no se desean recibir notificaciones sobre actualizaciones, este parámetro es necesario para utilizar *Provider.of()* dentro de *initState()*.

FutureProvider

FutureProvider es un tipo de proveedor que opera de manera similar a otros proveedores en Flutter, caracterizándose por su uso del parámetro *create*, el cual es obligatorio, este parámetro debe retornar un objeto *Future*, permitiendo así la integración de operaciones asíncronas dentro del árbol de widgets. Al resolver el *Future*, el valor resultante se pone a disposición de los widgets descendientes, facilitando la gestión de estados que dependen de datos obtenidos de manera asíncrona.

```
FutureProvider(
  initialData: null,
  create: (context) => createFuture(),
  child: <widget>,
);
Future<MyModel> createFuture() async {
  return Future.value(MyModel());
}
```

Un Future es útil cuando un valor no está disponible de inmediato, pero lo estará en el futuro, ejemplos de esto incluyen llamadas que solicitan datos de internet o lecturas de datos de una base de datos de forma asíncrona.

MultiProvider

MultiProvider es una clase diseñada para resolver situaciones en las que se requiere utilizar múltiples proveedores dentro de una aplicación. En lugar de anidar proveedores individualmente, lo que puede complicar la estructura del código y afectar la legibilidad, *MultiProvider* ofrece una solución más elegante y eficiente. Permite agrupar varios proveedores en un solo widget, facilitando así su manejo y configuración. Esta agrupación contribuye a mantener una estructura de código más limpia y organizada, optimizando el proceso de inyección de dependencias en la aplicación.

Entonces, esta clase responde a la pregunta, ¿Y si se necesita más de un proveedor? La respuesta inmediata es que se podría anidar los proveedores, de la siguiente manera:

```
Provider<MyModel>(
  create: (_) => Something(),
  child: Provider<MyDatabase>(
    create: (_) => SomethingMore()
    child: <widget>
  ),
);
```

Pero esto podría volverse confuso y dificultar la lectura y el mantenimiento del código, por lo tanto, en su lugar, se puede utilizar *MultiProvider* para crear una lista de proveedores y un único hijo, como se ve a continuación:

```
MultiProvider(
  providers: [
    Provider<MyModel>(create: (_) => Something()),
    Provider<MyDatabase>(create: (_) => SomethingMore()),
  ],
  child: <widget>
);
```

Dada la explicación teórica, es momento de poner en práctica al *Provider* en un ejemplo.

4.1.2 Cómo se usa Provider

Para comenzar a utilizar Provider, primero se lo debe añadir como dependencia en el proyecto donde se empleará, para que luego pueda emplearse el paquete cuya dependencia haya sido declarada. Para una comprensión más detallada de lo que es una dependencia, es útil referirse a lo que Cheng, p. (2019, p. 54) menciona:

Usar paquetes es la forma en que Dart gestiona las dependencias de proyectos. Flutter hereda la misma manera para la gestión de dependencias. Es posible que se haya visto conceptos similares en otras plataformas de programación. Para que la gestión de dependencias funcione, se necesita tener una forma de describir componentes compartibles y sus dependencias. También se necesita una herramienta para obtener las dependencias. [Traducción propia]

De manera precisa, en un proyecto Flutter se puede agregar una dependencia editando el archivo `pubspec.yaml`, en ese sentido, el siguiente ejemplo muestra dónde (sección *dependencies*) y cómo hacerlo (añadir `provider: ^x.x.x` donde debe ser reemplazado con la versión del paquete a emplear, normalmente la última versión). El código siguiente puede clarificar lo mencionado:

```
dependencias:  
  flutter:  
    sdk: flutter  
  provider: ^x.x.x # reemplaza x.x.x con la última versión
```

Una estructura de proyecto bien organizada facilita el mantenimiento y la escalabilidad del código. Por esta razón, se organizará el código del ejemplo siguiendo una arquitectura de “n capas”. A continuación, se muestra cómo se podría estructurar el proyecto:

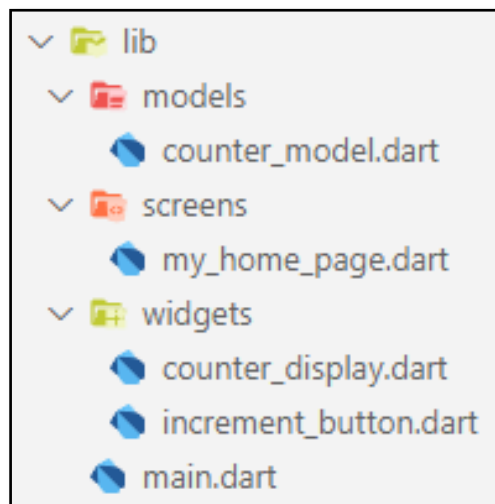


Figura IV.1. Estructura del proyecto contador con Provider

MANEJO DEL ESTADO CON PROVIDER

Ahora se procederá a trabajar en el código de cada uno de los archivos que forman la estructura presentada.

Inicialmente, se define un modelo que extienda *ChangeNotifier*, este modelo mantendrá el estado que se desea compartir, para ello, se edita el archivo `counter_model.dart`.

models/counter_model.dart

```
import 'package:flutter/material.dart';
class CounterModel extends ChangeNotifier {
  int _count = 0;
  int get count => _count;
  void increment() {
    _count++;
    notifyListeners();
  }
}
```

La clase *CounterModel* es una clase que extiende *ChangeNotifier*, esta es una clase que proporciona un mecanismo para notificar a los oyentes (por ejemplo, widgets) cuando un cambio ha ocurrido, esto es fundamental en el patrón *Provider* para la gestión del estado.

Dentro la clase se declara una variable `_count` que es privada (indicado por el guion bajo al inicio) y se inicializa con un valor de 0, esta variable mantiene el estado del contador.

Posteriormente, se define un getter, proporcionando un método conciso para retornar el valor de `_count`. Esto permite acceder al valor de `_count` desde fuera de la clase sin modificarlo directamente, manteniendo así el principio de encapsulación.

El método `increment()` es responsable de incrementar el valor de `_count` en uno cada vez que se llama, luego de modificar `_count`, se llama a `notifyListeners()`, esta es una función de *ChangeNotifier* que notifica a todos los oyentes (en este caso, los widgets que están consumiendo este modelo) que se ha producido un cambio. Como resultado, los widgets pueden actualizarse para reflejar el nuevo estado.

Ahora, para consumir el modelo, se envuelve la aplicación o el widget que necesite acceso al modelo con *ChangeNotifierProvider*. El siguiente código corresponde al archivo `main.dart`.

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import './models/counter_model.dart';
import './screens/my_home_page.dart';
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => CounterModel(),
```

```
        child: const MyApp(),
      ),
    );
  }
  class MyApp extends StatelessWidget {
    const MyApp({super.key});
    @override
    Widget build(BuildContext context) {
      return MaterialApp(
        title: 'Flutter Provider',
        theme: ThemeData(
          primarySwatch: Colors.orange,
        ),
        home: const MyHomePage(),
      );
    }
  }
}
```

Seguramente, gran parte del código mostrado le resultará familiar, sin embargo, el código novedoso es el que tiene que ver con el fragmento que utiliza *ChangeNotifierProvider*, que es parte del paquete *Provider*, este código está configurando el *Provider* para inyectar una instancia de *CounterModel* en el árbol de widgets.

Se examinarán cada una de las partes en detalle:

La clase *ChangeNotifierProvider* es un widget que proporciona una instancia de un *ChangeNotifier* a su árbol de widgets descendientes. En este caso, el *ChangeNotifier* es la clase *CounterModel*, esto significa que cualquier widget descendiente puede acceder a esa instancia de *CounterModel* y escuchar los cambios en ella.

El parámetro *create* de *ChangeNotifierProvider* toma una función que retorna la instancia del *ChangeNotifier* que quiere proveer. En este caso, la función crea una nueva instancia de *CounterModel*.

El guion bajo (“_”) es un convenio en Dart para indicar que el parámetro no se va a utilizar. Aquí, se refiere al contexto del *BuildContext*, que *ChangeNotifierProvider* pasa a la función *create*, pero en este caso no se requiere.

El parámetro *child* de *ChangeNotifierProvider* es el widget que se renderizará como hijo del *Provider*. En este ejemplo, el hijo es *MyApp*, que es el widget principal de la aplicación. Al usar *const* delante de *MyApp()*, se está creando una instancia constante de *MyApp*, lo que puede ayudar a mejorar el rendimiento al evitar reconstrucciones innecesarias.

A continuación, se procederá a trabajar en el archivo “my_home_page.dart”, que consiste en una pantalla con un texto en la parte central y un botón.

```
import 'package:flutter/material.dart';
import '../widgets/counter_display.dart';
import '../widgets/increment_button.dart';
class MyHomePage extends StatelessWidget {
  const MyHomePage({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Ejemplo con Provider'),
      ),
      body: const Center(
        child: CounterDisplay(),
      ),
      floatingActionButton: const IncrementButton(),
    );
  }
}
```

Este código no presenta ninguna novedad relacionada con el patrón Provider, por esa razón, se continuará trabajando en el widget *CounterDisplay()*, para ello se editará el archivo “counter_display.dart”.

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/counter_model.dart';
class CounterDisplay extends StatelessWidget {
  const CounterDisplay({super.key});
  @override
  Widget build(BuildContext context) {
    final counterModel = Provider.of<CounterModel>(context);
    return Text('Contador: ${counterModel.count}');
  }
}
```

Se observará que el nuevo código está relacionado con la línea:

```
final counterModel = Provider.of<CounterModel>(context);
```

Esta es una instrucción comúnmente usada cuando se trabaja con el paquete *Provider* para la gestión del estado, esta línea de código tiene un rol específico en el acceso al estado manejado por un *ChangeNotifier* (en este caso, *CounterModel*). A continuación, se procederá a desglosarlo para una mejor comprensión:

El método estático *Provider.of* proporcionado por el paquete *Provider* se utiliza para recuperar una instancia de un objeto (en este caso, *CounterModel*) que ha sido expuesto anteriormente en el árbol de widgets utilizando un *Provider* (como *ChangeNotifierProvider*).

El tipo genérico *<CounterModel>* se pasa a *Provider.of*, indicando así el tipo del objeto que se desea obtener, de esta manera, se especifica que la intención es acceder a una instancia de *CounterModel*.

El objeto *context* actúa como una referencia a la ubicación de un widget dentro del árbol de widgets. Al pasar *context* a *Provider.of*, se indica la intención de obtener una instancia de *CounterModel* que esté disponible en el árbol de widgets, en la ubicación actual o en alguna superior.

Por precisión, esta línea de código se usa dentro del widget para obtener acceso al modelo *CounterModel* que contiene el estado que el widget necesita presentar o modificar, en este caso, se emplea para acceder al contador.

Finalmente, se debe trabajar en el archivo “increment_button.dart”, que corresponde al botón diseñado para incrementar el contador.

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/counter_model.dart';
class IncrementButton extends StatelessWidget {
  const IncrementButton({super.key});
  @override
  Widget build(BuildContext context) {
    final counterModel = Provider.of<CounterModel>(context, listen: false);
    return FloatingActionButton(
      onPressed: counterModel.increment,
      child: const Icon(Icons.add),
    );
  }
}
```

En este caso se accede al modelo para emplear el método *increment* de la misma manera, es decir empleando:

```
final counterModel = Provider.of<CounterModel>(context, listen: false);
```

La diferencia está en la propiedad: *listen: false*, esta parte es significativa ya que indica que el widget que contiene esta línea de código no debe reconstruirse automáticamente cuando el modelo *CounterModel* notifique a sus oyentes (es decir, cuando se llame a *notifyListeners()* en el modelo).

En conclusión, *Provider* está construido sobre los fundamentos de los Inherited Widgets, el que se destaca por su eficacia y facilidad de uso en la gestión del estado a lo largo de aplicaciones más grandes y complejas. Al proporcionar una forma limpia y accesible de compartir el estado entre múltiples widgets, *Provider* no solo

simplifica el manejo del estado, sino que también promueve prácticas de programación más claras y mantenibles. Al dominar estos conceptos, se tiene las herramientas que posibilitan la creación de aplicaciones robustas y reactivas en Flutter, asegurando que la interfaz de usuario se mantenga actualizada con los cambios de estado y la lógica de la aplicación.

4.2 Ejercicio: empleo de Provider en la aplicación de lista de planes

En este ejercicio, se planteará una alternativa al manejo del estado desarrollado en el Capítulo 3, aplicando ahora *Provider* para el manejo de la funcionalidad de añadir y eliminar planes.

Se recuerda que en el ejercicio del Capítulo 3 se incluyeron algunos archivos y carpetas. Ahora para los cambios que se harán se incorporarán otros adicionales, siendo necesario mantener la siguiente estructura de archivos y carpetas.

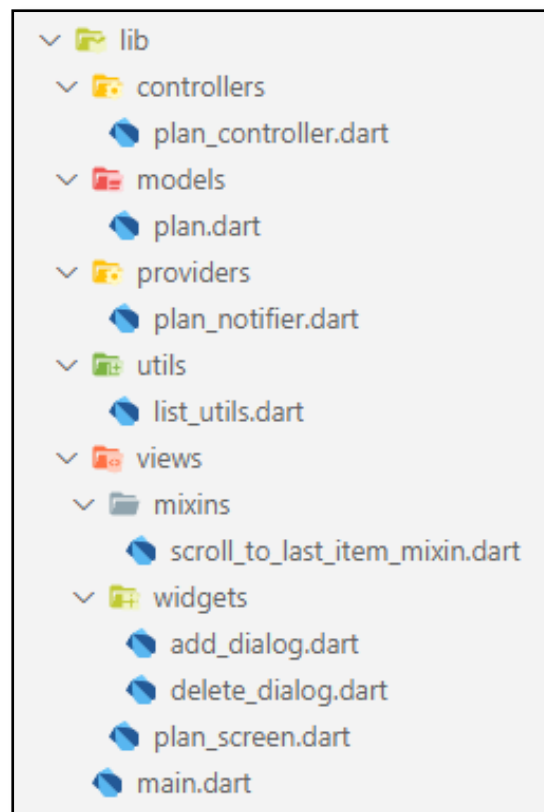


Figura IV.2. Estructura del directorio lib de la aplicación

Notará que la novedad está en la creación de las carpetas: “controllers”, “providers”, “utils” y “mixns”, además, los archivos: “plan_controller.dart”, “plan_notifier.dart”, “list_utils” y “scroll_to_last_item_mixin.dart”.

4.2.1 Instrucciones

En este ejercicio, se inicia trabajando en la creación de una utilidad ubicada en el archivo “list_utils.dart”, cuyo propósito es determinar si una cadena ya forma parte de una lista de cadenas. Posteriormente, se procede a desarrollar un controlador, ubicado en el archivo “plan_controller.dart”, encargado de gestionar la lista de

planes, la cual será administrada a través de un *Provider* escrito dentro el archivo “plan_notifier.dart”. Una vez implementado el *Provider*, es necesario realizar modificaciones en “main.dart” y luego en “plan_screen.dart” para permitir el manejo del *Provider*. Antes de abordar las modificaciones en “plan_screen.dart”, se desarrollará un *mixin* que permita desplazarse automáticamente al último elemento de la lista cada vez que se añade un nuevo plan.

El resultado al finalizar el ejercicio no presenta ninguna novedad en cuanto a diseño comparado con el Capítulo 3, ya que la diferencia radica en la gestión del estado, que ahora se gestionará con *Provider*, además de un adecuado manejo de los archivos considerando una arquitectura de n capas. Es así que, visualmente se tendrá lo siguiente:

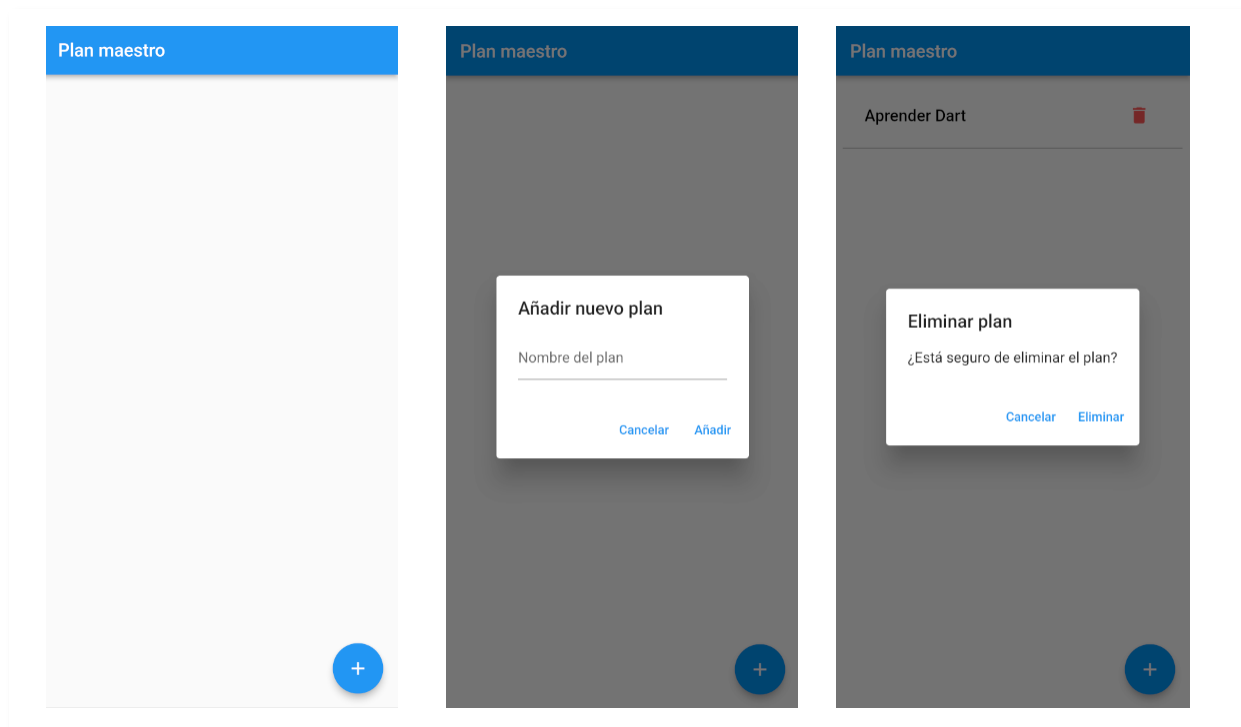


Figura IV.3. Pantallas de la APP lista de planes

Requisitos Previos.

Para iniciar el ejercicio, es necesario haber completado el ejercicio de la Sección 4 del Capítulo 3 y disponer de la aplicación resultante.

4.2.2 Creación de una utilidad

Un código diseñado para determinar si una cadena ya forma parte de una lista constituye una utilidad, en el sentido de que puede ser empleado en diversas funcionalidades para prevenir la repetición de cadenas en las interfaces. Por esta razón, se creará en un archivo independiente dentro de la carpeta “utils”, con el objetivo de ser accesible desde cualquier parte del proyecto.

utils/list_utils.dart

```
class ListUtils {
  static String checkForDuplicates(Iterable<String> items, String text) {
    final duplicatedCount = items.where((item) => item.contains(text)).length;
    if (duplicatedCount > 0) {
      text += ' ${duplicatedCount + 1}';
    } return text;
  }
}
```

El código presentado, hace una búsqueda de *text* dentro las lista *items*, en caso de que se encuentre la cadena en la lista, se generará una cadena señalando el número que le corresponde a la cadena por la cantidad de apariciones.

4.2.3 Implementación de un controlador

El código del controlador es el siguiente:

controllers/plan_controller.dart

```
import '../models/plan.dart';
import '../utils/list_utils.dart';
class PlanController {
  final _plans = <Plan>[];
  List<Plan> get plans => List.unmodifiable(_plans);
  void addPlan(String name) {
    if (name.isEmpty) name = "Plan";
    name = ListUtils.checkForDuplicates(plans.map((plan) => plan.name), name);
    _plans.add(Plan(name: name));
  }
  void deletePlan(Plan plan) {
    _plans.remove(plan);
  }
}
```

Este controlador permite el encapsulado de la lista de planes (*_plans*) brindando un getter que permite el acceso a la lista, pero sólo para lectura, cuidando así, que no se pueda alterar la lista desde una ubicación externa. Además, se implementan métodos exclusivos para añadir planes y eliminarlos de la lista. Además, se observa que, al momento de añadir un plan, se verifica que el nombre del plan no se repita.

4.2.4 Programación del Notifier

Es momento de programar el notificador.

El *Notifier* (*PlanNotifier*) es una clase que hereda de *ChangeNotifier* y emplea de manera interna un controlador (*_planController*) para acceder a los elementos de la lista de planes y a las funcionalidades de agregar y eliminar planes. Además, brinda dos getters, el primero para acceder a la lista de planes y el segundo para conocer la cantidad de planes que están en la lista.

providers/plan_notifier.dart

```
import 'package:flutter/material.dart';
import '../controllers/plan_controller.dart';
import '../models/plan.dart';
class PlanNotifier extends ChangeNotifier {
  final PlanController _planController;
  PlanNotifier(this._planController);
  List<Plan> get plans => _planController.plans;
  int get plansCount => plans.length;
  void addPlan(String name) {
    _planController.addPlan(name);
    notifyListeners();
  }
  void deletePlan(Plan plan) {
    _planController.deletePlan(plan);
    notifyListeners();
  }
}
```

Ahora se procederá a desarrollar las modificaciones en el archivo principal.

4.2.5 Modificación en el programa principal

Inicialmente se debe agregar la biblioteca “provider” al proyecto. Luego, las modificaciones están relacionadas con el consumo del *Provider* a través de *ChangeNotifierProvider*. Se recomienda consultar las secciones de explicación sobre *Provider* en caso de surgir dudas sobre su implementación.

main.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../controllers/plan_controller.dart';
import '../providers/plan_notifier.dart';
import '../views/plan_screen.dart';
void main() => runApp(const PlansApp());
class PlansApp extends StatelessWidget {
  const PlansApp({super.key});
  @override
```

```
Widget build(BuildContext context) {
  return ChangeNotifierProvider(
    create: (_) => PlanNotifier(PlanController()),
    child: const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: PlanScreen()
    ),
  );
}
```

Ahora, antes de modificar *PlanScreen*, se procederá a implementar el *mixin* para realizar *scroll* en la lista de planes.

4.2.6 Implementación de un Mixin

El código que se presentará a continuación, define un *mixin* llamado *ScrollToLastItemMixin*, diseñado para ser utilizado en estados (*State*) de widgets que extienden de *StatefulWidget*. Este *mixin* proporciona una funcionalidad específica para desplazarse automáticamente hacia el último elemento de una lista o un scrollable widget. Requiere que las clases que lo implementen proporcionen un *ScrollController* a través de un getter. La función *scrollToLastItem* utiliza este controlador para animar el desplazamiento hacia el punto más bajo posible (*maxScrollExtent*) del contenido scrollable, asegurando que el desplazamiento solo se ejecute si el controlador está asociado a algún widget (*hasClients*). La animación hacia el último elemento se realiza en un lapso de 500 milisegundos y utiliza la curva *easeOut* para suavizar el movimiento. Este *mixin* es útil para mejorar la interacción del usuario en aplicaciones que requieren visualización de listas o contenidos largos, facilitando el acceso al contenido más reciente con una animación fluida.

views/mixins/scroll_to_last_item_mixin.dart

```
import 'package:flutter/material.dart';
mixin ScrollToLastItemMixin<T extends StatefulWidget> on State<T> {
  ScrollController get scrollController;
  void scrollToLastItem() {
    if (scrollController.hasClients) {
      final position = scrollController.position.maxScrollExtent;
      scrollController.animateTo(
        position,
        duration: const Duration(milliseconds: 500),
        curve: Curves.easeOut,
      );
    }
  }
}
```

Finalmente se debe hacer modificaciones en el contenido de la clase *PlanScreen*, esto con la finalidad de emplear el *Provider* y el *mixín* ya desarrollados.

4.2.7 Modificación de PlanScreen

Se comenzará por aplicar el *mixín* a la clase correspondiente, mostrando al mismo tiempo las importaciones necesarias.

```
views/plan_screen.dart

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/plan_notifier.dart';
import '../models/plan.dart';
import '../widgets/add_dialog.dart';
import '../widgets/delete_dialog.dart';
import '../mixins/scroll_to_last_item_mixin.dart';
class PlanScreen extends StatefulWidget {
  const PlanScreen({super.key});
  @override
  State<PlanScreen> createState() => _PlanScreenState();
}
class _PlanScreenState extends State<PlanScreen> with ScrollToLastItemMixin {
  final ScrollController _scrollController = ScrollController();
  @override
  ScrollController get scrollController => _scrollController;
  @override
  void dispose() {
    scrollController.dispose();
    super.dispose();
  }
}
```

La parte central tiene que ver con la manera de indicar a la clase que se hará uso del *mixín*, esto tiene que ver con la declaración de la clase de estado:

```
class _PlanScreenState extends State<PlanScreen> with ScrollToLastItemMixin
```

Luego se procede con la creación de un objeto *ScrollController* de la siguiente manera:

```
final ScrollController _scrollController = ScrollController();
```

El acceso al *mixín* se hace sobre escribiendo el getter:

```
@override
ScrollController get scrollController => _scrollController;
```

Y también es necesario definir el método *dispose()* sobre escribiéndolo.

```
@override
void dispose() {
  scrollController.dispose();
  super.dispose();
}
```

Ahora es momento de trabajar en el consumo del *Provider*, para ello, primero de debe eliminar la lista *_plans* agregada en el capítulo anterior, la línea a eliminar es como la que sigue:

```
final List<Plan> _plans = [];
```

Una vez hecha la eliminación, el editor señalará los lugares donde se empleaba la lista como errores. Lo que se debe hacer para corregirlos es utilizar el *Provider* donde antes se empleaba la lista.

Tomando en cuenta que se deben hacer modificaciones a un código ya existente, se ve conveniente señalar como líneas sombreadas las nuevas líneas de código agregadas, con la finalidad de tener un contexto claro del resultado final:

Inicialmente se reflejan los cambios en el método *build*.

```
@override
Widget build(BuildContext context) {
  final planNotifier = Provider.of<PlanNotifier>(context);
  return Scaffold(
    appBar: AppBar(
      title: const Text('Plan maestro'),
    ),
    body: _buildList(planNotifier),
    floatingActionButton: FloatingActionButton(
      onPressed: () => _showAddPlanDialog(),
      tooltip: 'Agregar Plan', child: const Icon(Icons.add),
    ),
  );
}
```

Los cambios en el método *_buildList* de igual manera se reflejan con líneas sombreadas, eso es posible observarlo en el siguiente código:

```
Widget _buildList(PlanNotifier planNotifier) {
  return ListView.builder(
    controller: scrollController,
    itemCount: planNotifier.plansCount,
    itemBuilder: (context, index) {
      final plan = planNotifier.plans[index];
      return Container(
        margin: const EdgeInsets.all(8.0),
        padding: const EdgeInsets.all(8.0),
        decoration: const BoxDecoration(
          border: Border(
            bottom: BorderSide(
              color: Colors.black26, width: 1.0,
            ),
          ),
        ),
        child: ListTile(
          title: Text(
            plan.name,
            style: const TextStyle(fontWeight: FontWeight.w500, fontSize: 18.0),
          ),
          onTap: () { // Aquí iría tu código para manejar el onTap },
          trailing: IconButton(
            icon: const Icon(Icons.delete, color: Colors.redAccent),
            onPressed: () {
              _showDeletePlanDialog(planNotifier, plan);
            },
          ),
        ),
      );
    },
  );
}
```

También es necesario hacer cambios en el método `_showAddPlanDialog`, esos cambios se observan en el código siguiente.

```
void _showAddPlanDialog() {
  showDialog(
    context: context,
    builder: (BuildContext context) {
      return AddDialog(
        title: "Añadir nuevo plan",
        decoration: "Nombre del plan",
        onAdd: (String text) {
```

```
        final planNotifier = Provider.of<PlanNotifier>(context, listen: false);
        planNotifier.addPlan(text);
        scrollToLastItem();
    },
);
},
);
}
```

De la misma forma, los siguientes son los cambios en el método `_showDeletePlanDialog`.

```
void _showDeletePlanDialog(PlanNotifier planNotifier, Plan plan) {
  showDialog(
    context: context,
    builder: (BuildContext context) {
      return DeleteDialog(
        title: "Eliminar plan",
        content: "¿Está seguro de eliminar el plan?",
        onDelete: () {
          planNotifier.deletePlan(plan);
        });
    },
  );
}
```